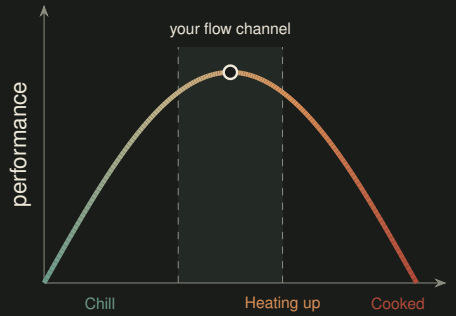
In Search of Sustainable Pace in Agentic Coding and Orchestration

Marinos Prevenios · Daphne Tsolissou

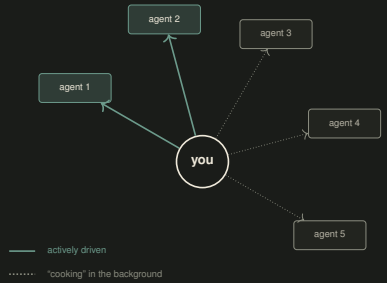


The problem

Software engineers stopped typing most of their code

- Coding agents produce code at machine pace, in concurrent sessions.
- The human role becomes supervision: dispatch tasks, monitor progress, judge partial results, re-engage failed or dormant threads.
- The natural incentive ("tokenmaxxing") is to run as many sessions as possible: tokens are cheap, attention is not.
- The binding resource is no longer typing speed. It is one person's attention.

Sheridan (1992), *Telerobotics, Automation, and Human Supervisory Control*: the operator who time-shares across semi-autonomous processes.



Other professions got there first

Automation does not subtract work. It converts producers into supervisors of parallel output, and the residual monitoring role gets harder, not easier.

Bainbridge (1983), “Ironies of Automation.”

Aviation	Cockpit automation moved pilots from flying to monitoring: vigilance decrement, “automation surprises.” Wiener & Curry (1980)
Anesthesiology	One anesthesiologist directs several operating rooms at once; higher ratios are associated with worse patient outcomes. Burns et al. (2022)
Air traffic	Controller workload scales with the number of aircraft held; decision support exists precisely to raise that count. Loft et al. (2007)
Radiology	Cross-sectional imaging multiplied images per study; an image to interpret every few seconds across a shift. McDonald et al. (2015)
Translation	Post-editing machine output turns translators into high-throughput reviewers; the result is measurably flatter. Krings (2001); Toral (2019)

What supervisory-control research already knows

- Fan-out: how many semi-autonomous units one operator can manage, bounded by *neglect time* and *interaction time*. Crandall & Cummings (2007); Cummings & Mitchell (2008), for UAV/robot operators.
- Past a personal fan-out limit, performance collapses non-linearly, often before the operator notices. Cummings & Mitchell (2008).
- Staying out of the loop degrades situation awareness and taxes re-engagement. Endsley & Kiris (1995); Eriksson & Stanton (2017).
- Monitoring an autonomous process is not idle time: it is effortful and stress-inducing. Warm, Parasuraman & Matthews (2008).
- Under concurrent load, operators systematically under-monitor: complacency. Parasuraman & Manzey (2010).

Supervising LLM coding agents is the same control problem in a new domain; the transfer is plausible, and testing it is part of this work.

In software, the effort shifts rather than disappears

- Software engineers spend ≈51.5% of session time in assistant-specific states: verifying suggestions, prompt-crafting, deferring thought. Mozannar, Bansal, Fourney & Horvitz (2024), CHI.
- Merely *informing* engineers that code is LLM-generated raises their measured NASA-TLX workload. Tang et al. (2024), IEEE VL/HCC.
- The interaction is bimodal: fast *acceleration* alternating with slow, validation-heavy *exploration*. Barke, James & Polikarpova (2023).
- A mixed-methods survey of 442 software engineers links GenAI adoption to burnout through intensified job demands. Feng, Afroz & Sarma (2026), ICSE-SEIS; Demerouti et al. (2001), Job Demands–Resources.

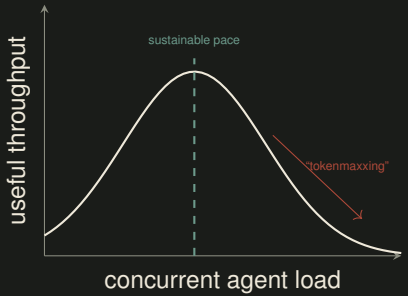
And oversight quality degrades quietly

- With an AI assistant, users wrote less secure code while being more confident it was secure. Perry, Srivastava, Kumar & Boneh (2023), ACM CCS.
- Professional engineers show automation complacency and rising reliance over the course of a single task. Qian & Wexler (2024), ACM IUI.
- In human code inspection, defect-removal effectiveness falls sharply past ≈200 lines reviewed per hour; review rate is the dominant controllable factor. Kemerer & Paulk (2009); the general speed–accuracy trade-off, Wickelgren (1977).

More parallel output funneled into the same reviewing hours means each change gets less scrutiny — reviewers are forced to go faster, compounding the complacency findings above.

The load is invisible from the inside

- Randomized trial: experienced open-source developers were ≈19% slower with AI tooling, yet believed it sped them up. Becker, Rush, Barnes & Rein (2025), METR; preprint.
- The perception–reality gap recurs in longitudinal IDE telemetry and in a 49,000-respondent survey. Sergeyuk et al. (2026), ICSE; Stack Overflow (2025).
- If a marginal session consumes attention faster than it yields verified output, adding sessions is counterproductive.



Inverted-U of performance under load: Yerkes & Dodson (1908); the flow channel, Csikszentmihalyi (1990).

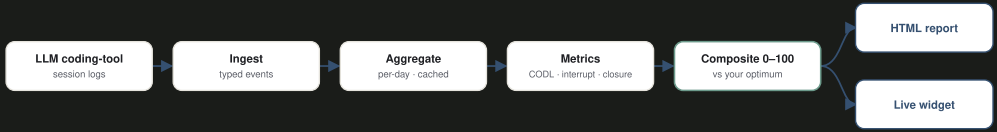
The problem, stated

- Kanban caps work-in-progress *not to slow delivery* but to protect flow and a sustainable pace. Anderson (2010); Beck (2000).
- Concurrent agent supervision is the same control problem with the WIP limit relocated into one human's working memory, where it is ordinarily invisible.
- The aim is not to use agents less: it is to find each person's operating band where throughput and supervision quality are jointly sustained.

What is missing is the signal: a private, longitudinal, research-grounded view of one's own supervision load, scored against one's *own* history, with a personally discovered optimum to steer toward.

The instrument

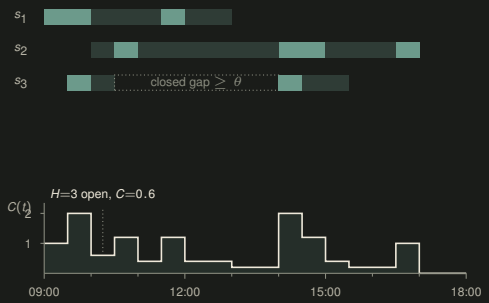
From logs already on disk to a daily score



Runs entirely on your machine — no network, no telemetry.

- Input: the session logs agent-coding tools already write locally (Claude Code, Codex CLI, . . .); one small adapter per tool maps them to typed events.
- Per-day aggregation with caching; three axes; a composite scored against a personal baseline and optimum.
- Python standard library only; no network I/O in the analysis; output is a self-contained HTML report plus a JSON sibling.

Axis 1 · CODL: engagement-weighted concurrency



- A session is foreground (e_s=1) within g=5 min of *your own* message.
- Otherwise, while live, it "cooks" at weight β=0.2; a closed tool counts nothing.
- Weighted concurrency $C(t)$ is charged per minute against working-memory capacity $\kappa \approx 4$, accumulating a capacity-dose toward a 240-min horizon.

Working memory holds ≈4 independent items: Cowan (2001). Fan-out and attention demand: Crandall & Cummings (2007).

Background sessions are not free: the case for $\beta > 0$

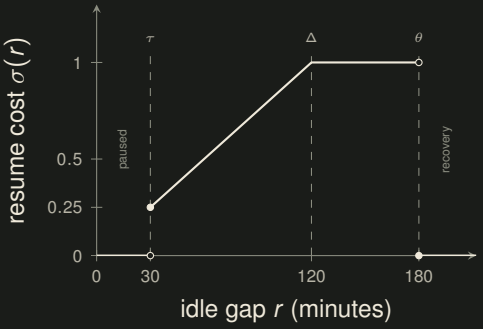
- A session left running while attention is elsewhere is the out-of-the-loop regime of supervisory control, not rest.
- Monitoring is effortful and stressful even when nothing is wrong. Warm, Parasuraman & Matthews (2008).
- Re-engaging costs: situation awareness decays out of the loop. Endsley & Kiris (1995); Eriksson & Stanton (2017).
- Under load, monitoring quietly degrades: complacency. Parasuraman & Manzey (2010).
- Magnitude borrowed from prospective memory: holding a pending intention slows the ongoing task by 15–20%, and open goals occupy memory until closed or planned. Smith (2003); Masicampo & Baumeister (2011).

Hence $\beta = 0.20$, the top of that band: a defensible, auditable prior, exposed in configuration rather than hard-coded, and a named target for calibration.

Axis 2 · Interruption Index

- Counts the events that pull attention: tool errors (weight 1.5) and cross-stream starts (weight 3.0), per work hour.
- Routine tool calls are deliberately not counted: while an agent works, the operator is legitimately waiting, and counting each call would inflate the rate by orders of magnitude.
- Normalization ceiling 10/hour, above the knowledge-work baseline of a working-sphere switch about every 11 minutes (≈5/hour). González & Mark (2004), CHI.
- Why it matters: interrupted work completes *faster* but at higher stress and effort; unfinished-task residue impairs the next task; cross-tool switches cost more than within-tool ones. Mark, Gudith & Klocke (2008); Mark, Gonzalez & Harris (2005); Leroy (2009); Wickens (2008).

Axis 3 · Closure Deficit: the cost of coming back

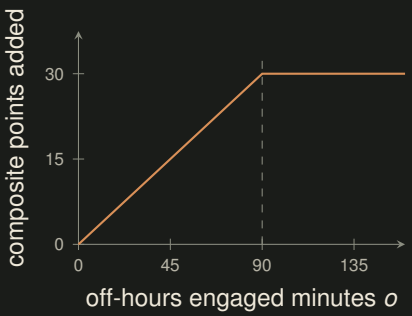


- A resume is a true-idle gap in one session, picked back up later: below $\tau = 30$ min it was merely paused; at $\theta = 3$ h the break counts as recovery, a fresh sitting.
- Cost saturates at $\Delta = 120$ min: the context has gone fully cold. Daily ceiling: about 4 cold reloads.

Suspended goals decay and must be reconstructed: Altmann & Trafton (2002). Resumption lag grows with interruption duration: Monk, Trafton & Boehm-Davis (2008). Interrupted programmers rarely resume coding within a minute: Parnin & Rugaber (2011). Closure as a recovery resource: Sonnentag & Fritz (2007).

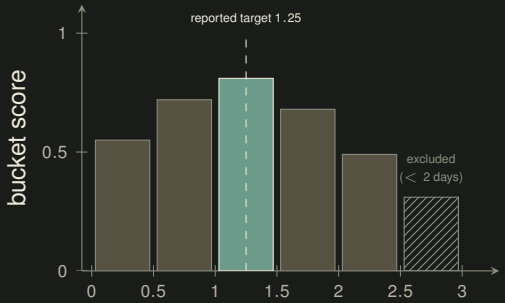
Composite score

- Composite = equal-weight mean of the three axes, on 0–100. Equal weights are the explicit null hypothesis, not a finding; calibrating them is named future work.
- Engaged minutes *outside your own inferred hours* add an additive toll: up to +30 points, saturating at 90 minutes.
- The boundary is asymmetric: evenings count in full, an early start gets a grace. The recovery literature locates the harm in failure to disengage at day's end. Sonnentag & Fritz (2007); Sonnentag, Binnewies & Mojza (2010); sustained activation, not peaks: McEwen (1998).
- Every day is colored against your own p50/p75/p90, never a population norm.



Anchored to interaction: a background session running while you are away adds nothing; only engaged off-hours minutes count.

Personal optimum

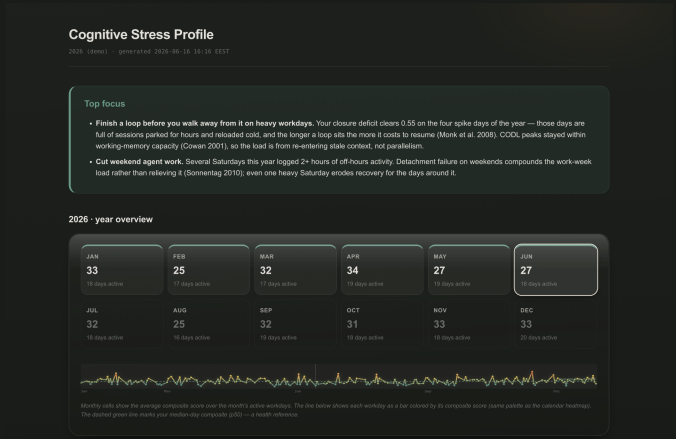


- Historical days are bucketed by mean weighted concurrency; each bucket is rewarded for closing loops and penalised for spilling past the window.
- The best bucket's midpoint becomes the flow-channel target: a line to steer toward, not a ceiling to stay under.
- This is a personal work-in-progress limit made *discoverable from one's own data*; it needs 14 active days, and shows "calibrating" before that.

Inverted-U of performance under load: Yerkes & Dodson (1908). The flow channel between boredom and anxiety: Csikszentmihalyi (1990).

The tool

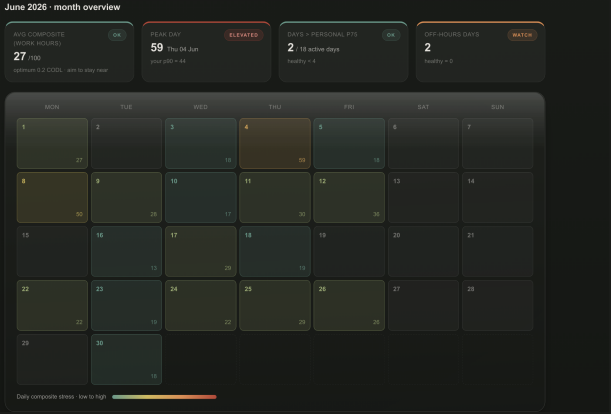
A year against your own baseline



- One self-contained HTML file; a JSON sibling for scripts and chat skills.
- Twelve monthly cells with active-day counts, plus a day-by-day strip against the personal median.
- “Top focus” guidance up front, each recommendation tied to the literature behind it.

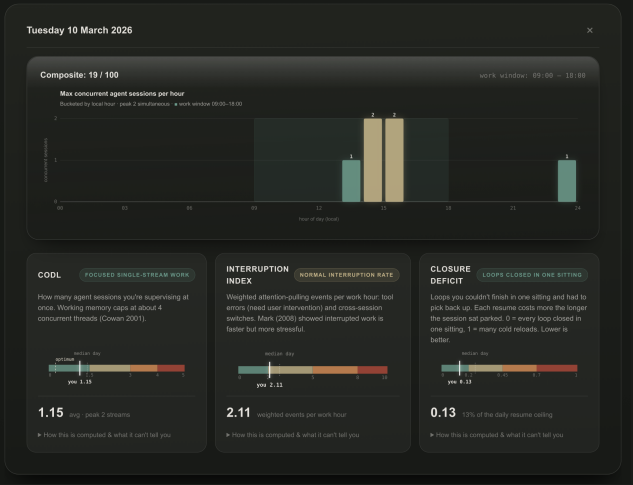
Shown throughout: the tool’s deterministic synthetic demonstration dataset; no operator’s activity is exposed.

Months as calendars, days against percentiles



- KPI tiles: average composite against the personal optimum, peak day, days above the personal p75, off-hours days.
- Calendar heatmap colored by each day's composite, relative to your own history.
- Zones use the same scale everywhere: report, JSON, and widgets can never drift apart.

Any day, drilled down



- Hourly concurrency inside the inferred work window.
- The three axes as tiles, each placed on its zone scale against a typical day marker.
- Every tile explains *how it is computed and what it cannot tell you*: the limitations ship inside the report.

Ambient, glanceable, always on



- One renderer produces a self-contained HTML card; thin hosts show it on KDE Plasma, any GTK desktop, macOS Übersicht, and Windows.
- Today, week, month, and year tabs; a compact mini variant.
- The card is where the optional felt-stress grade (0–2) is entered: one action, stored locally, used only for calibration.

Private by design

- Session transcripts routinely contain proprietary code and secrets; the generated report and JSON are treated as equally sensitive.
- The tool reads local files and writes local files: no telemetry, no remote rendering, nothing about the operator sent anywhere. Privacy is a hard invariant, not a feature toggle.
- One outbound request exists: the desktop companion asks, at most every six hours and switchable off, whether a newer version of the tool has been published. It discloses nothing in the asking.
- Intended use is individual self-assessment. The index is calibrated to one person's history; using an unvalidated load proxy to evaluate people would be statistically and ethically unsound, and the paper says so explicitly.

```
git clone https://github.com/sagium/ai-code-cognitive-stress
cd ai-code-cognitive-stress && python3 install.py && aicogstress --open
```

The research, honestly

What this is, and what it is not

It is

- an objective, behavioural taskload proxy, computed continuously from real sessions;
- longitudinal and per-person: only relative movement within one's own history is interpreted;
- fully open: every threshold is an explicit, cited, configurable prior.

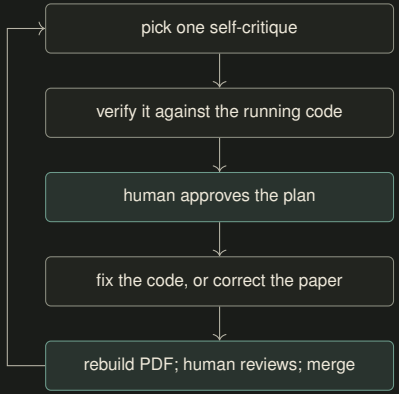
It is not

- a measure of *felt* workload: that is NASA-TLX; Hart & Staveland (1988).
- a burnout diagnosis: that is the MBI; Maslach & Jackson (1981).
- validated yet: thresholds are borrowed from adjacent literature, and no field data is reported; the illustrative example ships on synthetic sample data.

The paper attacks its own construct in an eleven-item threats section and states falsifiable predictions: if the three axes predict felt load no better than a stopwatch, the decomposition should be simplified away.

A paper that reviews its own code

- The paper and the implementation co-evolve: each self-critique is treated as an adversarial review of the code *and* of the premise behind an axis.
- A critique is first verified against the *running* code; then the code is improved, or the paper corrected, or both.
- Two human gates bracket every change: plan approval and merged-diff review. Tests must stay green.
- Reflexivity: a paper about supervising coding agents, produced by supervising coding agents.



The missing ingredient is a population

What pooled, anonymized exports enable

- Refit the axis *scales* (dose horizon, interruption ceiling) to how agent-coding engineers actually behave.
- Check the borrowed anchors: Cowan's $\kappa \approx 4$, the 10/hour ceiling, $\beta = 0.20$.
- With optional felt-stress grades: correlate each axis with felt load, and fit composite weights that must beat the equal-weight null *out of sample*.

What an export contains

- Derived daily metrics only: no code, no file paths, no usernames, no prompts.
- Timezone dropped; calendar dates randomly shifted.
- Optionally, your coarse 0–2 felt-stress grades.
- Written to a local file; you inspect it and upload it manually. The tool itself transmits nothing.

As contributions accrue, population baselines flow back into the tool, so every contributor sees how their patterns compare to the community.

Three ways to help

1. **Use it.** Clone the repo and ask your coding agent to install it: about a minute, and the report stays on your machine.
   `git clone https://github.com/sagium/ai-code-cognitive-stress`
2. **Fork it.** MIT-licensed, stdlib-only.
3. **Contribute your anonymized data.** Run `/contribute-data` in your coding agent: it writes the anonymized export locally and walks you through the manual upload.



https://github.com/sagium/ai-code-cognitive-stress